ScoreControl Backend: Progress and Next Week

Guy Nutman · October 30, 2026

Summary. Since last week, the database went from a design doc to a live Supabase database: all 11 tables (users, projects, files, commits, branches, pull requests) are deployed with security rules, and new users get their account record automatically. We applied every schema decision from the 10/22 meeting, added automated database tests, and built Google sign-in on the backend, which only needs the Google and Supabase dashboard setup to go live. Login works end to end with Ryan's pages, pending 4 small fixes before his branch merges. Next week: connect Google sign-in, build the API routes that let users create projects, upload scores, and save versions, and wire the UI to them (plan on page 2).

Area	Status	Where
Database (11 tables, RLS, user trigger)	Live on Supabase	PR #3, merged Oct 29
10/22 schema decisions	All applied	PR #3
Google sign-in	Backend built; needs dashboard setup and button	feat/google-sign-in
Login and signup pages	Working; 4 fixes before merge	ui/ui+auth (Ryan)
Project, upload, and commit API	Next up	Not started

1. The database is live on Supabase (PR #3, deployed Oct 29)

- Tables: users, projects, project_members, blobs (file info), score_metadata, commits, commit_parents, commit_files, branches, tags, pull_requests.
- RLS is on for every table, so the public Supabase key can't read or write anything directly; all access goes through our server.
- A trigger creates a users row automatically when someone signs up or picks a username, and deleting an account removes the user and their projects.
- Oliver's username migration was already on Supabase, so login needed nothing extra.
- The tables are empty until people sign up and the project API routes exist.

2. The 10/22 schema decisions are all applied

- 25 MB per file, enforced by the database; 100 files per project, enforced in the API.
- MusicXML is the default format; MuseScore and OTHER (PDFs, audio) are also allowed.
- `parsed_ast` caches the parsed score so the diff engine doesn't re-read files.
- AI columns: `commits.ai_summary` and `pull_requests.ai_conflict_notes`. Branches can be archived (hidden without deleting).
- Two technical changes from the spec, same behaviour: no direct foreign key from users to Supabase's auth table (Prisma breaks on it), and the trigger runs as a Prisma migration so local setup works.
- Testing: 10 database tests run on every npm test, migrations were verified on a real Postgres, and all 214 tests pass.

3. Google sign-in: backend done (branch feat/google-sign-in, pushed; PR next)

- A new route sends the user to Google with a one-time security check (PKCE); Oliver's existing route signs them in when they return.
- 6 tests, including that a sign-in code stolen from another browser is rejected.
- Still needed: Google Cloud OAuth client and Supabase provider setup, about 15 minutes (Guy); a "Continue with Google" button and a username form (Ryan).
- New Google users choose a username on first sign-in. A Google account with the same email as an existing password account is merged into it.

4. Login and frontend status

- Ryan's login and signup pages work end to end against the backend.
- His 4 fixes before merging: a lint error; the signup confirmation message never shows (`data.data.status`); show the API's specific error messages; read the username from `getCurrentUser()`.
- Email: Supabase's built-in sender works for team members only. A custom email provider (Resend's free tier plus a domain, about $10 a year) is needed before launch.

Housekeeping

- user-auth and setup are fully merged and can be deleted.
- The flaky auth test (Oliver) is still open.
- The AGENTS.md change log is up to date.

Next Week's Plan (Nov 2 to Nov 6)

Goal: by the end of next week, a signed-in user (password or Google) can create a project, upload a score, save versions, and see their history in the app.

#	Work	Owner	Depends on
1	Google sign-in live: dashboard setup, PR merged, button and username form	Guy, Ryan	-
2	Ryan's 4 login fixes, then his PR merged	Ryan	-
3	Storage bucket and shared database client	Guy	-
4	Project API routes: projects, upload and commit, history, download, branches	Guy, Oliver	3
5	UI for projects: list, create, upload, history pages wired to the API	Ryan, design team	2, 4
6	Logout and password reset pages	Ryan	2
7	CI on every PR and merge rules for main	Guy	-
8	Diff engine handoff: agree on the input format	Guy + diff team	-
9	LLM: commit message suggestion endpoint	Oliver	4
10	Shared test site on free hosting (e.g. Vercel)	Guy	1, 2

Owners are proposals; confirm them at the meeting.

1. Google sign-in live

- Google Cloud Console: OAuth client (Web application), redirect URI https://<project-ref>.supabase.co/auth/v1/callback; consent screen in Testing mode with team members as test users.
- Supabase: enable Google under Authentication > Sign In / Providers; add http://localhost:3000/auth/confirm to the redirect URLs.
- Ryan: a `Continue with Google` link to `/api/auth/oauth/google`, and a username form for users whose `/api/auth/me` returns username: null.
- Open and merge the feat/google-sign-in PR, then test with a real Google account.

2. Login fixes (Ryan)

- Pull main into ui/ui+auth; fix the lint error; read `data.data.status` after signup; show the API's error messages and field errors; use `getCurrentUser()` on the home page.
- Then open the PR. Merging it makes login usable from the real pages.

3. Storage bucket and database client

- A private Supabase Storage bucket for score files with a 25 MB limit; files stored at {projectId}/{sha256}.
- `src/lib/db.ts`: one shared Prisma client (using DATABASE_URL) for all API routes.

4. Project API routes

- Create and list projects; upload files and commit (hash, store, record the version, move the branch, all in one transaction).
- View history, download any version, create and archive branches; enforce 100 files per project.
- Every route checks who is signed in and whether they may access the project (owner, admin, editor, viewer). Tests for each route.
- Pull requests and merges come later, once the diff engine is connected.

5. UI for projects

- Pages for the project list, creating a project, uploading a score with a commit message, and viewing history.
- Built against the routes in item 4; the routes' request and response shapes get documented first so UI work can start in parallel.

6. Logout and password reset pages

- Logout button (POST /api/auth/logout), the forgot-password form, and the /reset-password page. The backend for all three already exists.

7. CI and merge rules

- A GitHub Actions workflow that runs lint, typecheck, and tests on every PR.
- GitHub settings: require 1 approval to merge into main, and delete branches automatically after merging. Delete the merged user-auth and setup branches.
- Fix the flaky auth test so CI results are reliable (Oliver).

8. Diff engine handoff

- Agree with the diff team on what they receive: the parsed score (parsed_ast), file paths, and which two or three versions to compare.
- Agree on what they return for merges and conflicts, so pull_requests.ai_conflict_notes can be filled in later.

9. LLM: commit message suggestions

- An endpoint that suggests a commit message from what changed; the user can edit it before saving to commits.message.
- Optional ai_summary for a longer "what changed" description. Decide on the model provider and budget (free tiers first).

10. Shared test site

- Deploy to free hosting (e.g. Vercel's hobby tier) so everyone tests the same URL instead of localhost.
- Needs that URL's APP_ORIGIN, redirect URL in Supabase, and environment variables set in the host. Email confirmation emails still only reach team members until we add a custom email provider.